

EDUGENIE

AI-Powered Learning Assistant Project Documentation

1. Introduction

EduGenie is an AI-powered educational learning assistant developed to help students learn academic concepts using Generative Artificial Intelligence. It provides Question and Answer, Topic Explanation, Quiz Generation, Text Summarization, and Personalized Learning Path features.

2. Objectives

The objectives are to provide quick academic answers, explain difficult concepts simply, generate multiple-choice questions, summarize educational content, recommend learning paths, provide an easy web interface, and demonstrate Generative AI in education.

3. Technology Stack

Frontend: HTML, CSS, JavaScript. Backend: Python and FastAPI. AI/LLM: Google Gemini. Optional local model: LaMini-Flan-T5. Other technologies: JSON, Jinja2, Pydantic, Uvicorn, and python-dotenv.

4. Architecture

Student → Web UI → JavaScript → FastAPI → Feature Module → AI Client → Gemini → Generated Response → Frontend. Feature modules include Q&A, Explanation, Quiz, Summary, and Learning Path.

5. Folder Structure

EduGenie/; main.py; config.py; ai_client.py; schemas.py; utils.py; qna.py; explanation_module.py; quiz_module.py; summary_module.py; learning_path.py; requirements.txt; .env; templates/index.html; static/style.css; static/app.js; tests/.

6. main.py

The main entry point of the FastAPI application. It configures CORS, serves static files, loads the HTML template, and defines GET /, GET /health, POST /qa, POST /explain, POST /quiz, POST /summarize, and POST /learn/recommendations.

7. config.py

Reads application configuration from the .env file, including the Gemini API key, model, local-model option, host, and port.

8. ai_client.py

Provides the common interface for Gemini. The generate_text() function sends prompts to the configured model and returns generated text.

9. schemas.py

Defines request and response structures with Pydantic. TextRequest validates input text, while quiz and feature response models define expected API data.

10. utils.py

Provides reusable JSON-processing functions. It removes Markdown code fences and parses AI-generated JSON safely.

11. qna.py

Answers student questions using a student-friendly educational prompt with simple language, concise explanations, and bullet points when useful.

12. explanation_module.py

Explains topics using the structure: Simple Definition, How It Works, Easy Example, and Important Points. It supports Gemini and an optional local model.

13. quiz_module.py

Generates exactly three multiple-choice questions, four options per question, one correct answer, and an explanation. The response is validated before returning it.

14. summary_module.py

Converts lengthy educational content into concise revision material while preserving important information and removing repetition.

15. learning_path.py

Creates a structured learning path with beginner, intermediate, advanced, project, resource, and four-week plan sections.

16. index.html

Provides the main EduGenie web interface, including task selection, input area, buttons, and result display.

17. style.css

Controls the application's visual design, including cards, buttons, inputs, quiz options, and responsive layout.

18. app.js

Handles frontend interaction, sends API requests, receives JSON responses, displays results, processes quizzes, and shows errors.

19. requirements.txt

Contains required Python packages such as FastAPI, Uvicorn, Jinja2, python-dotenv, Pydantic, Google GenAI, and Pytest.

20. .env

Stores environment configuration such as GEMINI_API_KEY, GEMINI_MODEL, USE_LOCAL_EXPLANATION, LOCAL_EXPLANATION_MODEL, HOST, and PORT. The API key must remain private.

21. Testing

Test the application through the web interface, the /health endpoint, and FastAPI interactive documentation. Run automated tests with: `python -m pytest`.

22. Installation

Create a virtual environment with `python -m venv .venv`. Activate it using `.venv\Scripts\Activate.ps1`. Install dependencies with `pip install -r requirements.txt`. Configure `.env` and run `python -m uvicorn main:app --reload`.

23. API Documentation

FastAPI provides interactive API documentation at <http://127.0.0.1:8000/docs>. It can be used to test the main API endpoints.

24. Workflow

Student → EduGenie Web UI → JavaScript → FastAPI → Feature Module → AI Client → Gemini → API Response → Web Interface → Student.

25. Advantages

AI-based academic assistance, simple explanations, automatic quiz generation, quick revision summaries, learning paths, modular architecture, REST APIs, and optional local AI support.

26. Future Enhancements

Future versions can add authentication, database integration, progress tracking, quiz history, PDF/document upload, voice input, multilingual support, teacher dashboard, and mobile applications.

27. Conclusion

EduGenie demonstrates practical Generative AI integration in education. It combines a web frontend, FastAPI backend, AI integration, educational modules, and REST APIs to provide an interactive learning assistant.